Comparative Analysis of Data-Driven Approaches for Hydrological Forecasting

A Preprint

Eldar Khuzin
Chair of Data Analysis
Moscow Institute of Physics and Technology
Moscow, Russia
khuzin.er@phystech.su

Novikov Ivan
Affiliation
Address
email

Abramov Dmitrii
Affiliation
Address
email

Abstract

In this article, we address the problem of equifinality in data-driven hydrological modeling — when different feature sets or model architectures yield similar predictive efficiency. We investigate the influence of static catchment features on predictions made by traditional models, recurrent neural networks, and transformers across various river basins in Eurasia. By analyzing feature importance and model behavior, we identify dominant predictors and assess the practicality of employing complex models in terms of interpretability and computational cost. This approach provides insights into the trade-offs between model complexity and predictive robustness in the presence of equifinality.

Keywords Flood discharge · Rainfall-runoff modeling · More

List of suggested changes

1 Introduction

Add models from the problem statment section as “Quantile Random Forest (QRF)”, “Gradient-Boosted Trees (GBT)” and so on.

2 Problem Statement

Notation

- Let $y \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$ denote target; $x \in \mathbb{R}^{n \times d}$ and $x_i \in \mathbb{R}^d$ denote d features associated with y_i, where n is the number of samples;
- Let the dataset be $\mathcal{D} = \{ (x_i, y_i) \mid i \in \overline{1, n} \}$ — set of n pairs for features and targets.
- Let the models class be denoted as $\mathcal{F}$;
- Let $\hat{y}_i \coloneqq f(x_i)$, $f \in \mathcal{F}$ denotes estimation of y_i based on x_i;
- Let assume $\hat{y} \coloneqq (\hat{y}_1, \ldots, \hat{y}_n) = (f(x_1), \ldots, f(x_n))$.

2.1 Optimization statement

In general, in each experiment we find the optimal model f^* s.t.

$$f^* = \arg\min_{f \in \mathcal{F}} \left(\mathcal{L}(f, \mathcal{D}) + \Omega(f) \right), \tag{1}$$

where:

- $\mathcal{L}(f, \mathcal{D})$ denotes the empirical loss of model f on dataset $\mathcal{D}$;
- $\Omega(f)$ is a regularization term penalizing model complexity.

Let $\ell(\cdot, \cdot)$ denotes the actual loss function, so $\mathcal{L}(f, \mathcal{D}) = \ell(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i)$.

In our work we choose ℓ as a mean absolute percentage error (MAPE), so 1 can be rewriten as

Link to article and explanation why MAPE

$$f^* = \arg\min_{f \in \mathcal{F}} \left(\sum_{i=1}^{n} \left| \frac{y_i - f(x_i)}{y_i} \right| + \Omega(f) \right). \tag{Opt. task}$$

2.2 Model's descriptions

2.2.1 Regression trees

Add link to article, where trees were presented

In some experiments we use regression trees as base predictors. In general, regression tree is binary tree, which can be written as

$$f(x) = \sum_{l=1}^{L} \mathbf{1}_{\{ x \in R_l \}} \cdot \omega_l, \tag{2}$$

where:

- R_l, $l \in \overline{1, L}$ form a partition of $\mathbb{R}^d$ into disjoint sets;
- ω_l are constant weights on tree leaves.

To perform the partitioning into R_l, each node of the binary tree acts as a simple predicate of the form $x^{(j)} \leq t$, used to select between the left and right child. Here, $x^{(j)}$ denotes a coordinate in the features $\mathbb{R}^d$ space, and t is a *threshold*.

Add a picture of three

To select a threshold t and the corresponding feature index j at each node, the algorithm maximizes an objective function known as the *information gain (IG, $\mathcal{IG}$)*. We are going to use the same IG as Chen and Guestrin 2016. Let assume $\hat{y}_i^{(0)}$ as initial prediction (constant):

Think about initial value.

$$\mathcal{IG}(t,j) = \frac{1}{2}\left[\frac{\left(\sum_{i\in\mathcal{I}_L} g_i\right)^2}{\sum_{i\in\mathcal{I}_L} h_i + \lambda} + \frac{\left(\sum_{i\in\mathcal{I}_R} g_i\right)^2}{\sum_{i\in\mathcal{I}_R} h_i + \lambda} - \frac{\left(\sum_{i\in\mathcal{I}} g_i\right)^2}{\sum_{i\in\mathcal{I}} h_i + \lambda}\right] - \gamma,$$

where:

- $\mathcal{I}_L = \{\, i : x_i^{(j)} \le t\} -$ indices, which are going to be in the left child if node use threshold t and feature j;
- $\mathcal{I}_R = \{\, i : x_i^{(j)} > t\} -$ same, but in the right child;
- $g_i = \frac{\partial \ell(y_i,\hat{y}_i)}{\partial \hat{y}_i}\Big|_{\hat{y}_i=\hat{y}_i^{(0)}}$ and $h_i = \frac{\partial^2 \ell(y_i,\hat{y}_i)}{\partial \hat{y}_i^2}\Big|_{\hat{y}_i=\hat{y}_i^{(0)}}$ are the first and second derivatives of the loss;
- λ is the L_2 regularization weight on leaf scores;
- γ is the complexity penalty for adding a new leaf.

In other words, we find a best split in node for Opt. task where complexity penalizes is $\Omega(f) = \gamma L + \frac{1}{2}\lambda\|w\|_2^2$

Then, once the tree structure is fixed, we assign each leaf l the weight

$$\omega_l = -\frac{G_l}{H_l + \lambda}, \quad \text{where } G_l = \sum_{i\in R_l} g_i, H_l = \sum_{i\in R_l} h_i.$$

2.2.2 Random Forest

Random Forest (RF) is an ensemble of B regression treesBreiman 2001, each trained on a bootstrap draw of the training set. At every split, instead of scanning all d features, only a random subset of $m_{\text{try}} \ll d$ candidates is used. Thus, each tree $f^{(i)}$ is both bagged and feature-subsampled, and the RF prediction is

$$f(x) = \frac{1}{B}\sum_{I=1}^{B} f^{(i)}(x). \tag{3}$$

This dual randomness (bagging + feature subsampling) (1) lowers variance by averaging many decorrelated trees, and (2) secure from overfitting despite fully grown leaves.

Primary hyperparameters:

- B — number of trees;
- m_{try} — (features per split);
- "Bootstrap sample size" — n draws with replacement.

Breiman 2001 showed that as $B \to \infty$, RF variance go to zero if trees correlation is lowest.

2.2.3 Gradient Boosting

Gradient Boosting (also called Gradient Boosting Machine, GBM) Friedman 2001 builds an additive ensemble of B regression trees by successive error correction. Starting from an initial guess $f^{(0)}(x)$, each new tree $h^{(b)}$ is fit to the negative gradient (residuals) of the loss ℓ with respect to the current model. Formally, GBM is

$$f_{(B)}(x) = f^{(0)} + \nu\sum_{i=1}^{B} f^{(i)}(x), \tag{4}$$

where

- ν — is a learing rate hyperparameter;
- B — number of trees;

- $f^{(b)}$ — regression tree;

and each of the tree is trained to predict the errors of the previous state model $f_{(B-1)}$:

$$f^{(B)} = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \ell(y_i - f_{(B-1)}(x_i), f(x_i)) \tag{5}$$

2.2.4 SARIMA

Seasonal Autoregressive (AR) Integrated Moving Average (MA) (SARIMA) is the extention of ARIMA model (combination of SARIMA).

Let's denotes

- L — back-shift (lag) operator, means $L\, y_t = y_{t-1}$;
- $a(L) = 1 - a_1 L - \cdots - a_p L^p$ (AR polynomial);
- $b(L) = 1 - b_1 L - \cdots - b_q L^q$ (MA polynomial);
- d — number of non-seasonal differences needed for stationarity.

ARIMA (p, d, q)

$$(1 - L)^d\, y_t = \mu + \frac{b(L)}{a(L)}\, \varepsilon_t, \tag{6}$$

Add seasonality $\Rightarrow$ **SARIMA** $(p, d, q) \times (P, D, Q)_s$

$$(1 - L)^d (1 - L^s)^D\, y_t = \mu + \frac{b(L)\, B(L^s)}{a(L)\, A(L^s)}\, \varepsilon_t, \tag{7}$$

where

- s — seasonal period;
- (P, D, Q) — AR, differencing, and MA orders of the *seasonal* component:
  - $A(L) = 1 - A_1\, L^s - A_2\, L^{2s} - \cdots - A_P\, L^{Ps}$;
  - $B(L) = 1 - B_1\, L^s - B_2\, L^{2s} - \cdots - B_Q\, L^{Qs}$.

2.3 Evaluation Metrics Used

Notation

- n is the total number of observations;
- y_i denotes the value for the i-th observation;
- $\widehat{y}_i$ denotes the model estimation value for the i-th observation based on features x_i;
- $\overline{y}$ is the sample mean.

Root Mean Squared Error (RMSE)

$$\text{RMSE}\,(y, \widehat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2}. \tag{RMSE}$$

Mean Absolute Error (MAE)

$$\text{MAE}\,(y, \widehat{y}) = \frac{1}{n} \sum_{i=1}^{n} |y_i - \widehat{y}_i|\,. \tag{MAE}$$

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE}\,(y, \widehat{y}) = \frac{1}{n} \sum_{i=1}^{n} \left| \frac{y_i - \widehat{y}_i}{y_i} \right| \cdot 100\%. \tag{MAPE}$$

Nash-Sutcliffe Efficiency (NSE) Originally developed in hydrology, the NSE evaluates the predictive power of hydrological models.

$$\text{NSE}\left(y, \widehat{y}\right) = 1 - \frac{\sum_{i=1}^{n}\left(y_i - \widehat{y}_i\right)^2}{\sum_{i=1}^{n}\left(y_i - \overline{y}\right)^2}. \tag{NSE}$$

Interpretation:

- NSE $= 1$ indicates a perfect match between model and observations.
- NSE $= 0$ implies that the model is no better than using the mean of the observed data.
- NSE < 0 suggests the model predictions are worse than the mean.

3 Methodology

3.1 Correlation analysis

Describe the idea

4 Computational experiment

Software

All experiments were conducted in Python 3.12 using the following open-source libraries:

- `scikit-learn` (version *.*.*) for implementation of Random Forest models Pedregosa et al. n.d.
- `XGBoost` (version *.*.*) for gradient boosting models Chen and Guestrin 2016.
- `Dask` (version) for parallel and distributed computation Rocklin 2015.
- `NumPy` and `Pandas` for data manipulation and preprocessing.
- `Matplotlib` and `Seaborn` for visualization.

Experiments were executed on a machine . The complete source code and environment configuration files are available at: `https://github.com/intsystems/2025-Project-188`.

describe the machine

Datasets

todo

Map

todo

Time series

todo

Static features

todo

Symbol	Formula	Description
$\overline{x}$	$\frac{1}{n}\sum_{i=1}^{n} x_i$	Sample mean of the values $x_1, \ldots, x_n$

Table 1: Summary of frequently used symbols and their meanings.

4.1 Experiment 1. Random Forest

4.2 Experiment 2. GBRT

4.3 Experiment 3. SARIMAX

4.4 Experiment 4. LSTM

5 Results

6 Discussion

7 Conclusion

Notations

References

Breiman, Leo (Oct. 1, 2001). "Random Forests." In: *Machine Learning* 45.1, pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324. URL: https://doi.org/10.1023/A:1010933404324 (visited on 04/24/2025).

Chen, Tianqi and Carlos Guestrin (Aug. 13, 2016). "XGBoost: A Scalable Tree Boosting System." In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16: The 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco California USA: ACM, pp. 785–794. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785. URL: https://dl.acm.org/doi/10.1145/2939672.2939785 (visited on 04/16/2025).

Friedman, Jerome H. (Oct. 1, 2001). "Greedy Function Approximation: A Gradient Boosting Machine." In: *Ann. Statist.* 29.5. ISSN: 0090-5364. DOI: 10.1214/aos/1013203451. URL: https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full (visited on 04/16/2025).

Pedregosa, Fabian et al. (n.d.). "Scikit-Learn: Machine Learning in Python." In: *MACHINE LEARNING IN PYTHON* ().

Rocklin, Matthew (2015). "Dask: Parallel Computation with Blocked Algorithms and Task Scheduling." In: Python in Science Conference. Austin, Texas, pp. 126–132. DOI: 10.25080/Majora-7b98e3ed-013. URL: https://doi.curvenote.com/10.25080/Majora-7b98e3ed-013 (visited on 04/16/2025).